מבוא לבינה מלאכותית - תרגול 6.

רגב יחזקאל אימרה.

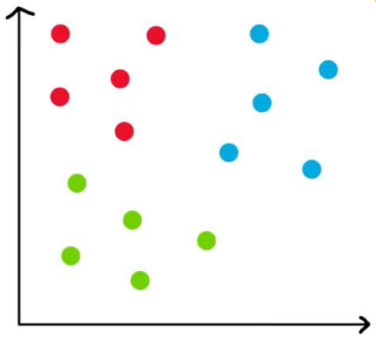
1 אלגוריתם KNN.

עד כה, הגדרנו מודל באותה צורה:

מגדירים בעיה $\leftarrow$ בונים מודל $\leftarrow$ מגדירים את פונקציית $loss$ $\leftarrow$ מבצעים אופטימיזציה על את הפרמטרים של המודל.

מה אם אנחנו לא נגדיר מודל?

שאלה 1.1. בהינתן הנקודות הבאות:



איך אנחנו יכולים לסווג נקודה חדשה?

תשובה: פשוט נסווג אותה לפי התוויות של השכנים שלה!

הרעיון הוא מאוד פשוט: נקבע מראש קבוע k שלפיו נסווג את הנקודה.

אלגוריתם 2.1. בהינתן נקודה חדשה:

1. חשבו את המרחק שלה לכל נקודה אחרת.
2. מצאו את k הנקודות הכי קרובות אל הנקודה שלנו.
3. סווגו את הנקודה החדשה אל המחלקה שבה נמצאים הכי הרבה שכנים שלה

וזהו! בנינו את אלגוריתם KNN!

הערה 3.1. בשלב (2), אנחנו צריכים לחשב את המרחק בין 2 דגימות, זה יכול להיות מאוד מסובך אם:

1. לא כל הדגימות שלי מקבלות אותו משקל.
2. יחידות המידה של הדגימות שונות.
3. יש לנו דגימות שלא תורמות לסיווג.

לכן אנחנו צריכים להגדיר **מטריקה** על הדגימות שלנו.

שאלה 4.1. האם כל מטריקה תעבוד?

תשובה: לא! המטריקה שאנחנו משתמשים בה צריכה לבטא קרבה בין הדאטה שלנו. האלגוריתם מניח שנקודות קרובות אחת לשנייה דומות מבחינת סיווג, לכן אנחנו צריכים להביא לו מטריקה שמשקפת את זה.

שאלה 5.1. האם אנחנו יכולים להשתמש ב-KNN לעוד משימה?

תשובה: כן! אם אני מעוניין במשימת רגרסיה, אני יכול פשוט לחזות את y שלי להיות ממוצע ערכי ה-y של k הדגימות הכי קרובות שלי ב-x

אלגוריתם 6.1. בהינתן נקודה חדשה:

1. חשבו את המרחק שלה לכל נקודה אחרת.
2. מצאו את k הנקודות הכי קרובות אל הנקודה שלנו $\{y_{i_j}\}_{j=1}^{k}$.
3. החזירו $\hat{y} = \frac{1}{k}\sum_{j=1}^{k} y_{i_j}$.

יתרונות וחסרונות של אלגוריתם KNN:

יתרונות:

1. פשוט למימוש - רק צריך מטריקה מתאימה ולבחור את k (לא צריך $loss$, ולא GD).
2. לא צריך לאמן את המודל.

3. יכול לעבוד על דאטה לינארי, לא לינארי, וככל שיש יותר דאטה הפלט יותר מדויק.
4. עובד לסיווג ורגרסיה.

חסרונות:

1. צריך לעבור על כל הדאטה - $O(Nd)$ להחזרת פלט (d = כמות הפיצ'רים, N = כמות הדגימות שלי).
2. יקר בזיכרון - צריך לשמור את כל הנקודות, בעוד שבעבר היה אפשר לשמור כמות נמוכה יחסית של פרמטרים.
3. בערכי k קטנים, $outliars$ יכולים להשתלט על הדגימה, ובערכי k גדולים, קבוצות קטנות מדי יכולות להיעלם.
4. צריך שהדאטה יהיה מאוזן (אני לא יכול לתת אותו משקל לגובה 1.8 מטר ול-$1,000,000$ ש"ח), אבל אין איך לעשות רגולריזציה.
5. בממדים גבוהים מאוד (d ענקי), מרחק בין 2 נקודות מאבד משמעות.
6. רגיש לפיצ'רים שיכולים להיות לא שימושיים - פיצ'ר אחד יכול להרוס את כל המרחק.

איך אפשר לתקן את זה: המרחק (האוקלידי) בין 2 נקודות הוא

$$d(x,z) = \|x-z\|_2 = \sqrt{(x-z)^T(x-z)}$$

ואם אנחנו רוצים למשקל את המרחק הזה, נוכל להגדיר

$$d_W(x,z) = \sqrt{(x-z)^T W(x-z)}$$

על מנת ללמוד קשרים בין $x_i - z_i$ ובין $x_j - z_j$. כדי להבטיח שהמרחק אינו שלילי, נדרוש ש-W תהיה חיובית חצי-מוגדרת, למשל על ידי $W = L^T L$. כך נקבל פסאודו-מטריקה, והיא מטריקה ממש אם L בעלת דרגת עמודות מלאה כלומר לא נלמד d_W אלא נלמד

$$d_L(x,z) = \|L(x-z)\|_2$$

החיסרון כאן זה שהפעם כן צריך לאמן את W, ניתן לקוראים בבית כתור תרגיל לחשוב על פונקציית $loss$ מתאימה.

שאלה 7.1. האם אנחנו יכולים להשתמש ב-KNN לעוד משימה חוץ מחיזוי y?

תשובה: כן! אנחנו יכולים להשתמש באלגוריתם כדי להשלים דאטה חסר:

נניח שיש לי דאטה $x_i = \left(x_i^{(1)}, \ldots, x_i^{(d)}\right), y_i$ אבל עבור דגימה כלשהי, אחד מהפיצ'רים לא נדגם כמו שצריך לדוגמה:

	$x^{(1)}$	$x^{(2)}$	$x^{(3)}$	$x^{(4)}$	y
(x_1, y_1)	17	3	0	1	0
(x_2, y_2)	-9	-	0	2	1
(x_3, y_3)	4	2	1	3	1
(x_4, y_4)	7	0	1	3	0

אזי $x_2^{(2)}$ חסר. נוכל לחשב את ה-k דגימות הכי קרובות אליו (מבחינת ערכי x שלא חסרים) $\{x_{i_j}\}_{j=1}^k$ ואז להשלים

$$x_2^{(2)} = \frac{1}{k}\sum_{j=1}^{k} x_{i_j}^{(2)}$$

2 קביעת היפרפרמטרים ו-$Cross-validation$.

אני עכשיו רוצה לבחור בין כמה מודלים. נניח המשימה שלי היא סיווג בינארי (למרות שהרעיון זהה לכל משימה אחרת).

בעבר פשוט חילקנו את הדאטה ל-$train\backslash test$ ולקחנו את המודל עם ה-AUC הכי גבוה.

$Data$

$Train$	$Test$

הבעיה היא שלחלק את הדאטה שלנו זה משהו שאנחנו לא רוצים, היינו רוצים שיהיה לנו $test$ כמה שיותר גדול כדי שתהיה לנו הערכה אמפירית כמה שיותר אמיתית לאיכות המודל (גם ככל שהדאטה יותר קטן, לחלק את הדאטה שלי עוד יותר מכניס שאלה של מה היה קורה אם הייתי דוגם אחרת ל-$test$). אני לא יכול לבדוק את הדאטה שלי על ה-$train$ כי אז זה לא ישקף לי כמה המודל מכליל, אבל אני לא יכול לבדוק את המודל שלי על כל הדאטה כי אז אני לא אאמן את המודל שלי. מה נעשה?

אנחנו יכולים לחלק את הדאטה שלנו ל-5 קבוצות הנקראות $Folds$

$Data$

$Fold\ 1$	$Fold\ 2$	$Fold\ 3$	$Fold\ 4$	$Fold\ 5$

ואז אם בעבר הייתי משתמש ב־$Fold$ החמישי בשביל ה־$test$, אני עכשיו יכול להשתמש בכל ה־$Folds$ שלי. איך?

פשוט נעבור על כולן, ובכל פעם נבחר אותה להיות ה־$test$ שלנו.

עכשיו אימנו 5 מודלים ויש לנו חמישה AUC, מה נעשה איתם? נחשב להם ממוצע וסטיית תקן, כאשר הממוצע יעריך לנו את ה־AUC האמיתי של המודל וסטיית התקן תאמר לנו כמה לסמוך על הממוצע.

לתהליך קוראים $Cross-validation$ או בקיצור CV ויראה ככה:

$Data$

$model\ 1$	$Train$	$Train$	$Train$	$Train$	$Test$
$model\ 2$	$Train$	$Train$	$Train$	$Test$	$Train$
$model\ 3$	$Train$	$Train$	$Test$	$Train$	$Train$
$model\ 4$	$Train$	$Test$	$Train$	$Train$	$Train$
$model\ 5$	$Test$	$Train$	$Train$	$Train$	$Train$

ככה יצא שאני בוחן את המודל שלי על כל הדאטה, בלי לאמן ולבחון על אותה דגימה בו זמנית.

כשאני מחלק את הדאטה שלי ל־5 קבוצות קוראים לתהליך $5\ Fold\ Cross\ Validation$, ובכללי כשמחלקים ל־$k$ קבוצות קוראים לתהליך $k-Fold\ Cross-Validation$.

אני עכשיו רוצה למצוא את ההיפרפרמטרים הכי טובים למודל שלי (יכול להיות d דרגת הפולינום ברגרסיה פולינומיאלית או λ קבוע הרגולריזציה או k כמות השכנים שאני משתמש בהם ב־KNN) איך נעשה את זה?

הפתרון נורא פשוט: נגדיר טווח לכל היפרפרמטר ונדגום אותם משם, נעריך כמה טוב המודל שלנו בזכותם ובסוף נבחר את הצירוף הכי טוב

דוגמה 1.2. נניח אני מאמן מודל רגרסיה פולינומיאלית, ששם אני יכול לקבוע את:

- d - מעלת הפולינום.
- λ_1 - עוצמת רגולריזציית l_1.
- λ_2 - עוצמת רגולריזציית l_2.
- k - כמה שכנים אני לוקח כדי להשלים איתם ערכים חסרים.

אזי נגדיר לכל אחד תחום שאנחנו רוצים למצוא היפרפרמטרים מתוכו:

- $d \in \{2,\dots,9\}$.
- $\lambda_1 \in \{0, 0.001, 0.01, 0.1, 1\}$.
- $\lambda_2 \in \{0, 0.001, 0.01, 0.1, 1\}$.
- $k \in \{5, 10\}$

נשים לב שאם הייתי מחפש בכוח על כל הצירופים האפשריים, הייתי עובר על 180 קומבינציות פה, ואם הייתי קובע $\lambda_1 \in [0,1]$ היו לי אֵ אפשרויות להיפרפרמטרים (הרבה אפשרויות), לכן נדגום ברנדומליות נניח 5 פעמים ונקבל

d	λ_1	λ_2	k	MSE
6	0.1	0.1	10	2.155543
4	0.1	0.01	10	27.248841
7	1	0.001	5	5.280761
4	0.001	0.1	5	2.726469
2	0	1	10	7

אזי קל וחומר שניקח את הצירוף הראשון של $d=6, \lambda_1=0.1, \lambda_2=0.1, 10$ ונדווח שהמודל הכי טוב שלי קיבל MSE של 2.155543.

שאלה 2.2. האם אפשר להשתמש ב־CV כדי לקבוע היפרפרמטרים?

תשובה: ברור שכן, ואפילו מומלץ. נשתמש ב־$k-Fold\ CV$ על ה־$Train$ (אחרי שפיצלנו ל־$train\backslash test$), נדגום את ההיפרפרמטרים שלנו ונקבל הפעם ממוצע לכל דגימה, וניקח את ההיפרפרמטרים עם הממוצע הכי גבוה (בתנאי שאין לו סטיית תקן מטורפת, כמובן), ואז נעריך את המודל שלנו על ה־$test$.